COL11D1

LECTURE - 8

AUGUST 7, 2026

LINKED LIST

WHAT IS THE OUTPUT?

class → items, add

```
class Bag:
    items = []
    def add(self, x):
        self.items.append(x)

a = Bag()
b = Bag()
a.add("apple")
print(b.items)
```

THOUGHT EXPERIMENT

→ Open a text file with a million characters in an editor

→ Assume editor models it as a sequence of characters

→ Put the cursor at the front & type one letter

Sequence → array → O(n)

Today's lecture: how to enable such editor?

Last Class: Sequence ADT

A finite, ordered collection, addressed by ~~position~~ index

```
length()        how many elements
get(i)          the element at position i
set(i, x)       overwrite position i
insert(i, x)    put x at i, sliding the rest right
delete(i)       remove at i, sliding the rest left
```

→ can be implemented using arrays.

Array-Based Implementation

```
class ArraySequence:
    def __init__(self):
        self._A = []
```

→ list data type

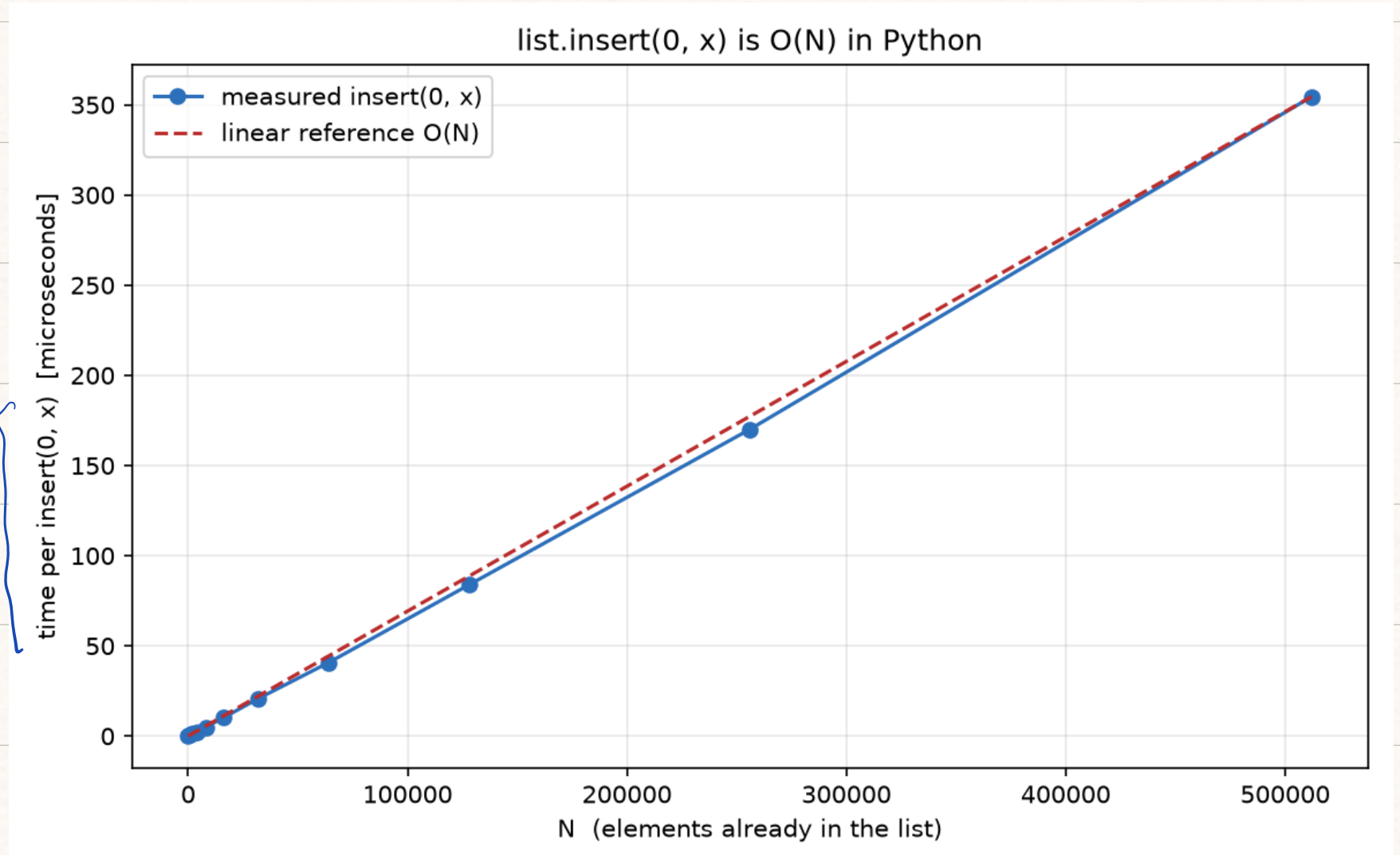
```
O(1) ← def length(self):  return len(self._A)
O(1) ← def get(self, i):  return self._A[i]
O(1) ← def set(self, i, x): self._A[i] = x
```

```
O(n-i) ← def insert(self, i, x): self._A.insert(i, x)
         def delete(self, i):    del self._A[i]
```

Insert Is Really A Loop

```
def insert(self, i, x):
    self._A.append(None)           # room for one more
    for j in range(len(self._A) - 1, i, -1):
        self._A[j] = self._A[j - 1]  # slide each tail element right
    self._A[i] = x
```

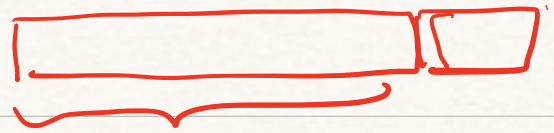
Insert Is Really A Loop



WHY MUST INSERT SHIFT EVERYTHING ?

- Data is stored in contiguous memory slot
- To insert, you must create a gap
  - existing element at i needs to move forward
and so on...

SECOND LIMITATION : THE NEED TO RESIZE

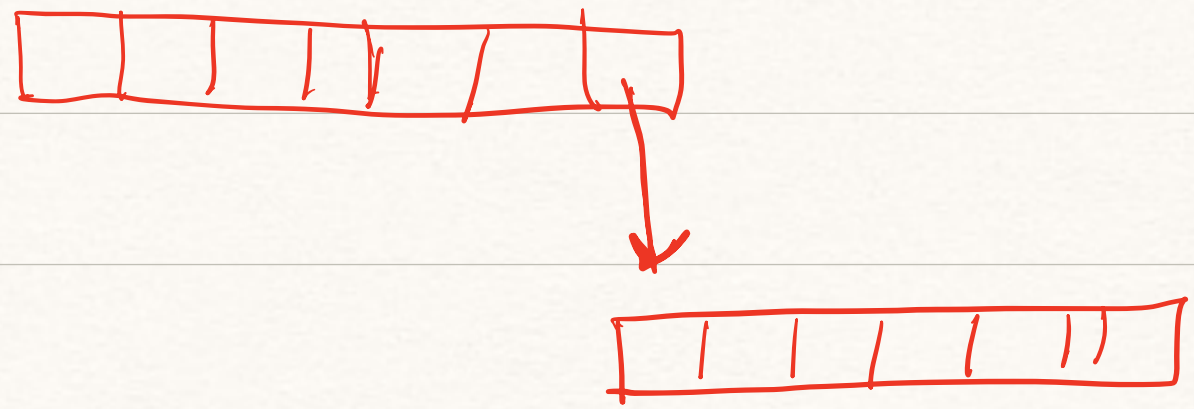


→ contiguous block has fixed capacity

→ when it fills, the only option

allocate a bigger block, copy everything across

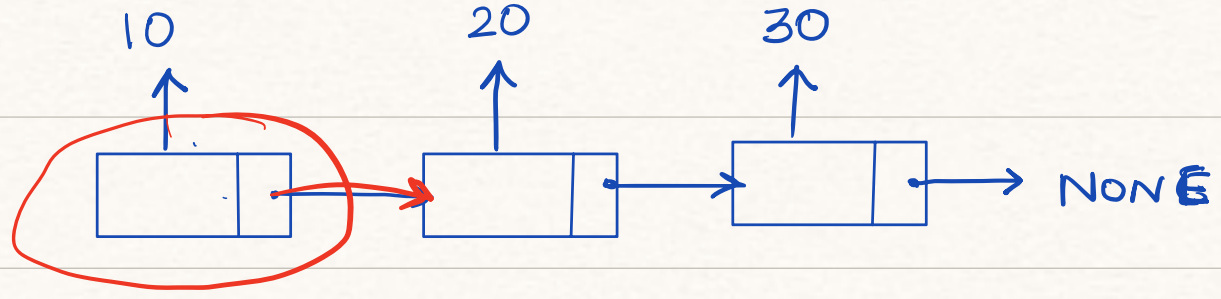
A contiguous array can never grow in place



Assumption we broke: array consists of contiguous memory

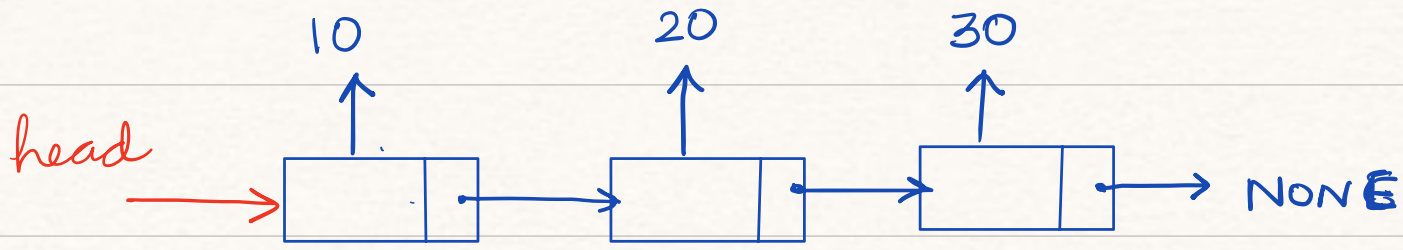
AN ALTERNATE DATA STRUCTURE

- what if elements did NOT have to be contiguous
- each element is stored in a node that also stores the location of next node



AN ALTERNATE DATA STRUCTURE

- what if elements did NOT have to be contiguous
- each element is stored in a node that also stores the location of next node



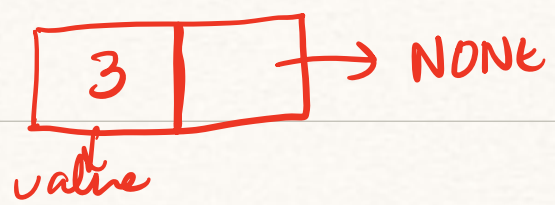
- Only need to know about head to navigate the list
- Last node's next is None
- Nodes may sit anywhere in memory

LINKED LIST

Node Class

```python
class Node:
    def __init__(self, value, next=None):
        # the element itself
        self.value = value
        # reference to the NEXT node -- or None
        self.next  = next
```

Node (3)

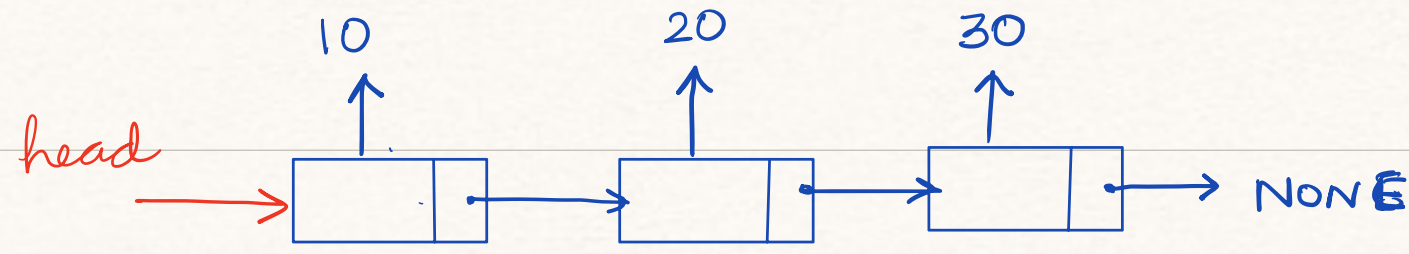


Linked List Container

```
class LinkedList:
    def __init__(self):
        self._head = None
        self._n = 0

    def length(self):
        return self._n
```

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next  = next
```



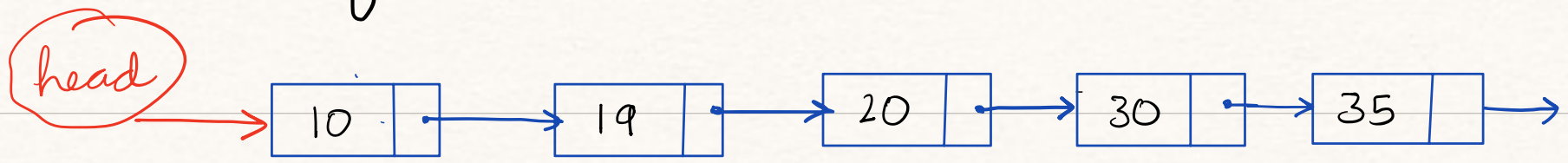
- whole list is head pointing at the first node
- can reach any node from there

Implementing Sequence ADT Using Linked List

```
length()        how many elements
get(i)          the element at position i
set(i, x)       overwrite position i
insert(i, x)    put x at i, sliding the rest right
delete(i)       remove at i, sliding the rest left
```

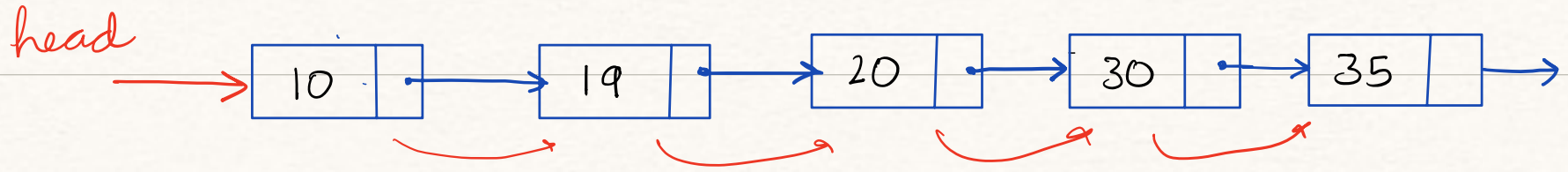
GET / SET : NO ARITHMETIC, ONLY WALKING

get (4) : To read the 4th element, need to traverse four elements



GET / SET : NO ARITHMETIC, ONLY WALKING

get(4): To read the 4th element, need to traverse four elements




```python
def _node_at(self, i):          # 0 <= i < n
    node = self._head
    for _ in range(i):
        node = node.next        # one hop along the chain
    return node

def get(self, i): return self._node_at(i).value
def set(self, i, x): self._node_at(i).value = x
```

RANDOM ACCESS IS GONE

Time Complexity

get(i) : O(i)

set(i,x) : O(i)

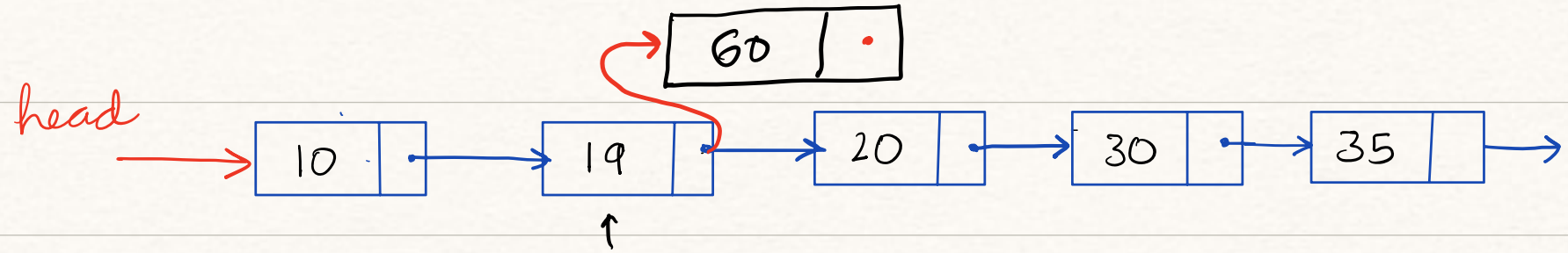
→ Traversing to the ith node : $O(i)$

reading the value at that node : O(1)

Where do we gain?

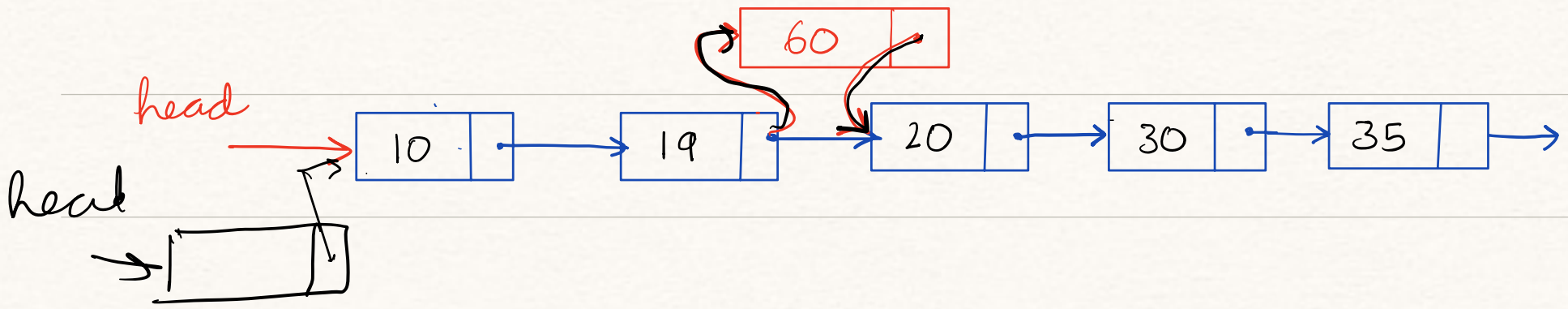
INSERT

Suppose we want to insert 60 at 2nd position



INSERT

Suppose we want to insert 60 at 2nd position



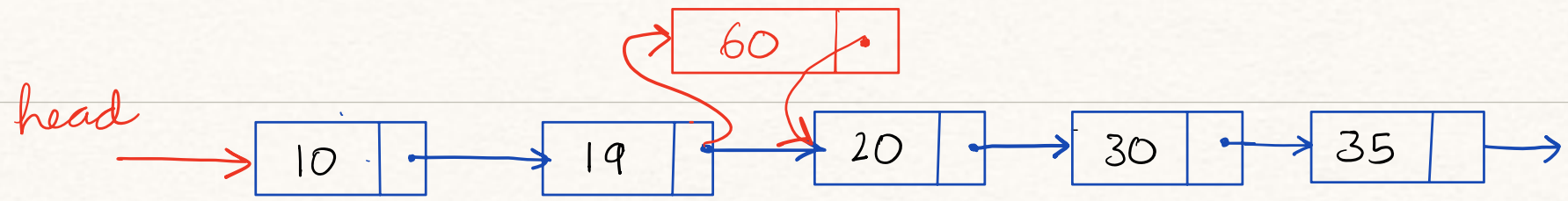

```
def insert (self, i, x):
    new = Node (x)
    if i > 0:
        prev = self._node_at(i-1)
        prev.next = new
        new.next = prev.next
    else:
        new.next = self.head
        self.head = new
```

new.next = prev.next

prev.next = new ✗

INSERT

Suppose we want to insert 60 at 2nd position




```
def insert(self, i, x):          # 0 <= i <= n
    if i == 0:
        self._head = Node(x, self._head)
    else:
        prev = self._node_at(i - 1)
        prev.next = Node(x, prev.next)      # splice in
    self._n += 1
```

INSERT: TIME COMPLEXITY

insert (i, x)

```
def insert(self, i, x):          # 0 <= i <= n
    if i == 0:
        self._head = Node(x, self._head)
    else:
        prev = self._node_at(i - 1)
        prev.next = Node(x, prev.next)     # splice in
    self._n += 1
```

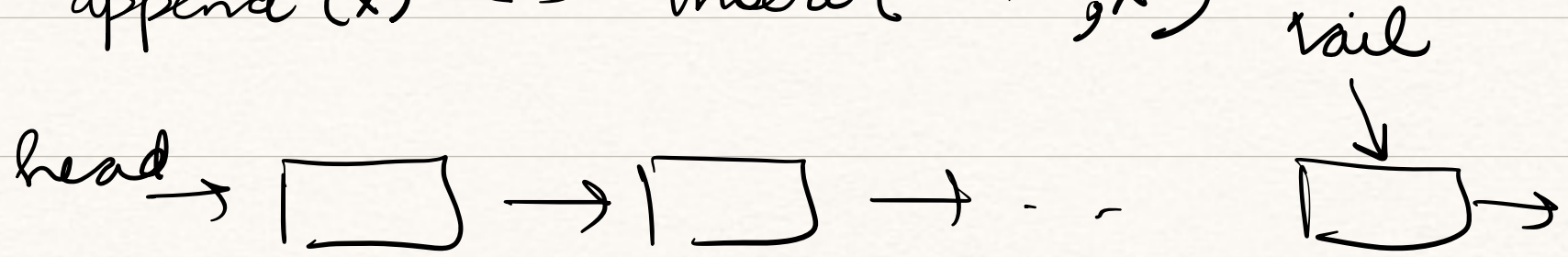
O(i):

traverse : O(i) → Best case: insert in the beginning

insert : O(1) worst case: insert at the end

APPEND OPERATION :

append (x) ⟷ insert (n , x)



traverse to the end.

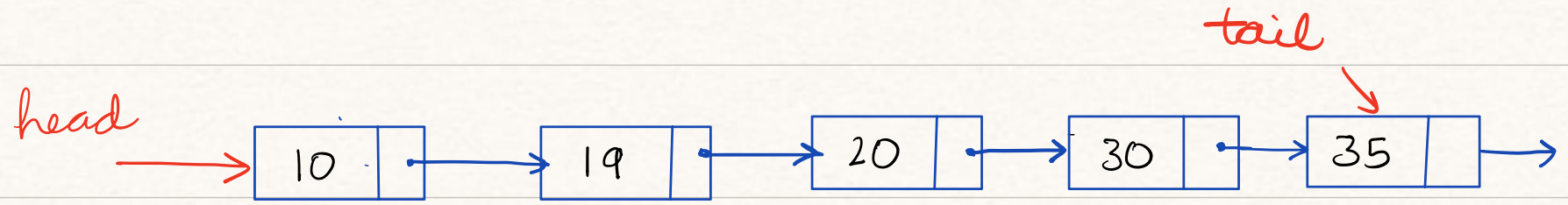
insert

APPEND OPERATION :

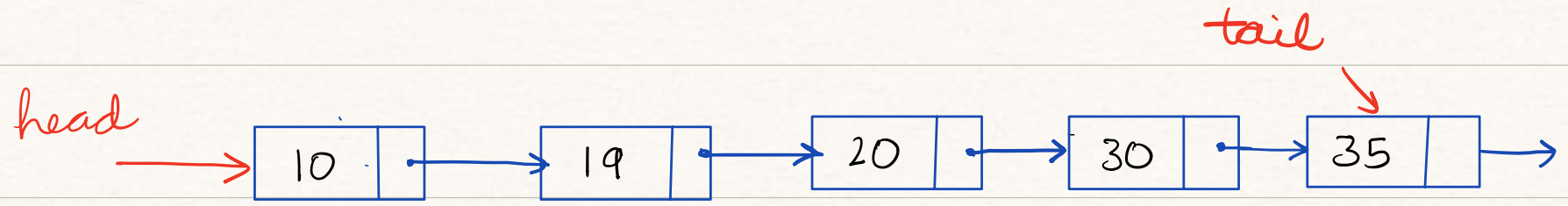
append (x) $\leftrightarrow$ insert (n, x)

Can we make it faster ?

LINKED LIST WITH TAIL



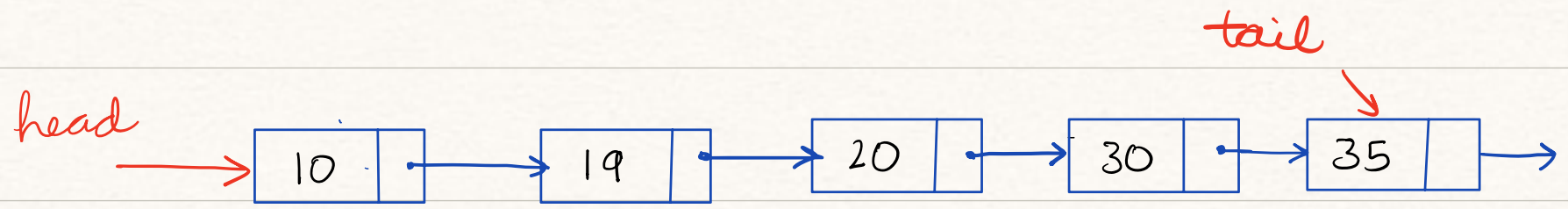
LINKED LIST WITH TAIL : CODE




```
class LinkedList:
    def __init__(self):
        self._head = None
        self._tail = None
        self._n = 0

    def length(self):
        return self._n
```

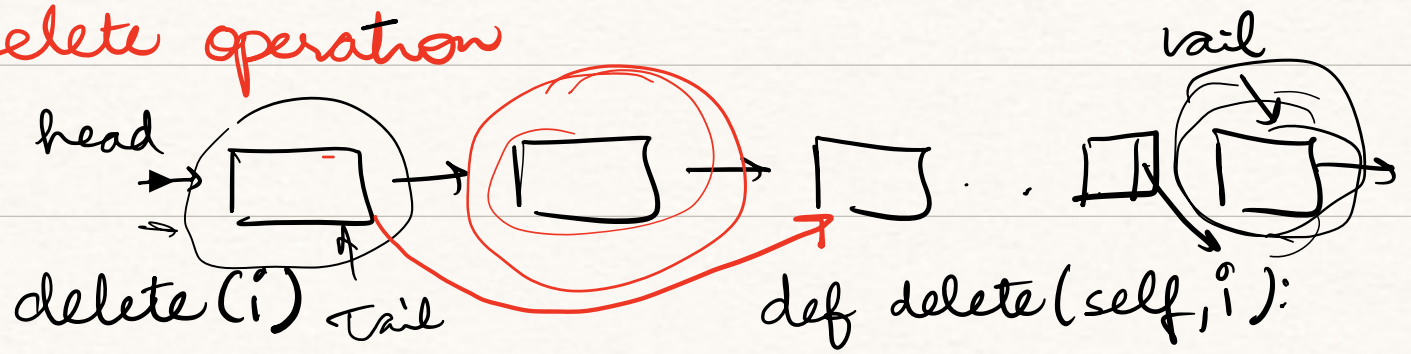
LINKED LIST WITH TAIL : CODE




```
def insert(self, i, x):          # 0 <= i <= n
    if i == 0:                              # front: O(1)
        self._head = Node(x, self._head)
        if self._n == 0:                    # was empty -> tail is the new node too
            self._tail = self._head
    elif i == self._n:                      # back: O(1) via cached _tail
        self._tail.next = Node(x)
        self._tail = self._tail.next
    else:                                   # middle: O(i) walk, then splice
        prev = self._node_at(i - 1)
        prev.next = Node(x, prev.next)
    self._n += 1

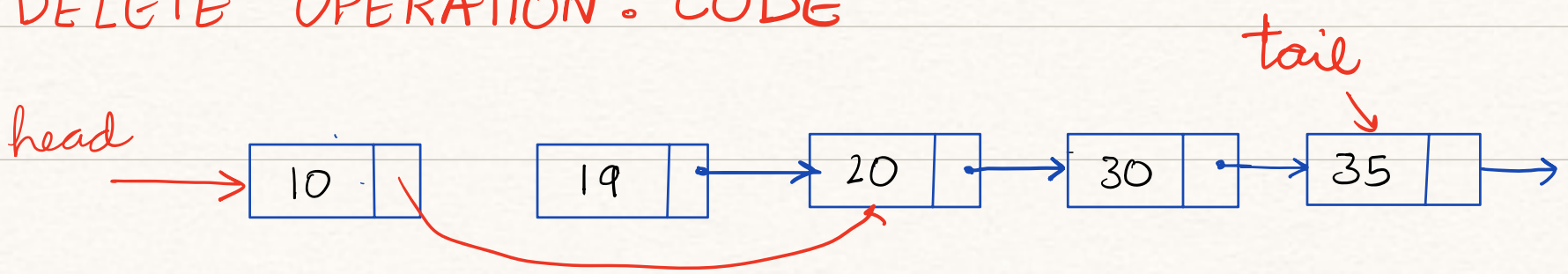
def append(self, x):
    self.insert(self._n, x)     # append == insert at the end
```

Delete operation




```
def delete(self, i):
    if i == 0:
        self._head = self._head.next
        if self._n == 1:
            self._tail = None
    elif i == self._n - 1
        prev = self._node_at(i-1)
        prev.next = None
        self._tail = prev
    else:
        prev
        prev.next = prev.next.next
self._n -= 1
```

DELETE OPERATION : CODE




```
def delete(self, i):                # 0 <= i < n
    if i == 0:
        self._head = self._head.next
        if self._head is None: self._tail = None
    else:
        prev = self._node_at(i - 1)
        prev.next = prev.next.next      # splice the node OUT
    self._n -= 1
```

SCORE CARD

Operation	Array	Linked list
length ()	O(1)	O(1)
get(i)	O(1)	O(i)
set (i,x)	O(1)	O(i)
insert (i,x)	O(n-i)*	O(i)
delete (i)	O(n-i)*	O(i)

* amortized

BACK TO THE THOUGHT EXPERIMENT

→ Open a text file with a million characters in an editor

→ Assume editor models it as a sequence of characters

→ Put the cursor at the front & type one letter

Linked list

↳ Is the cursor an index?

set (i, 'c')

Back To The Thought Experiment

→ Open a text file with a million characters in an editor

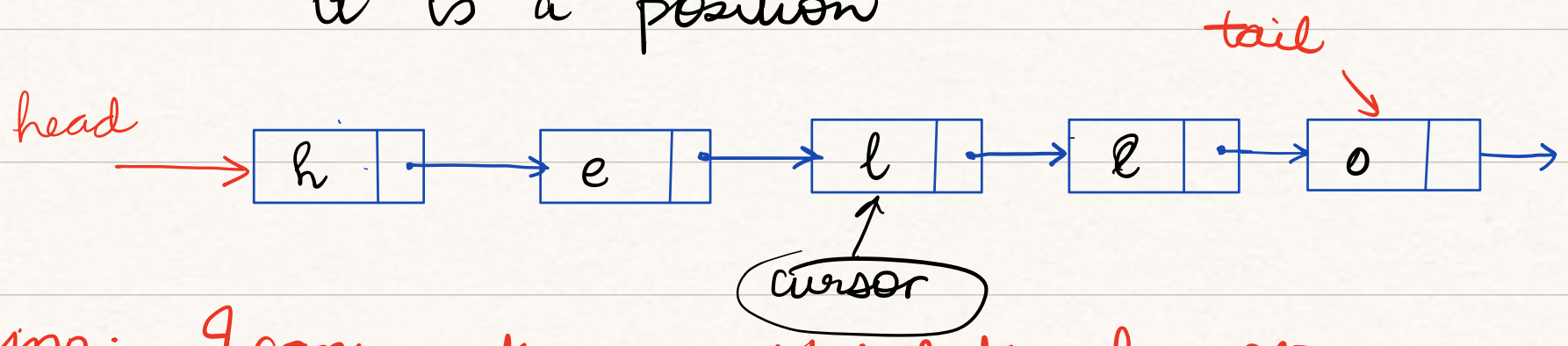
→ Assume editor models it as a sequence of characters

→ Put the cursor at the front & type one letter

Is the cursor an index?

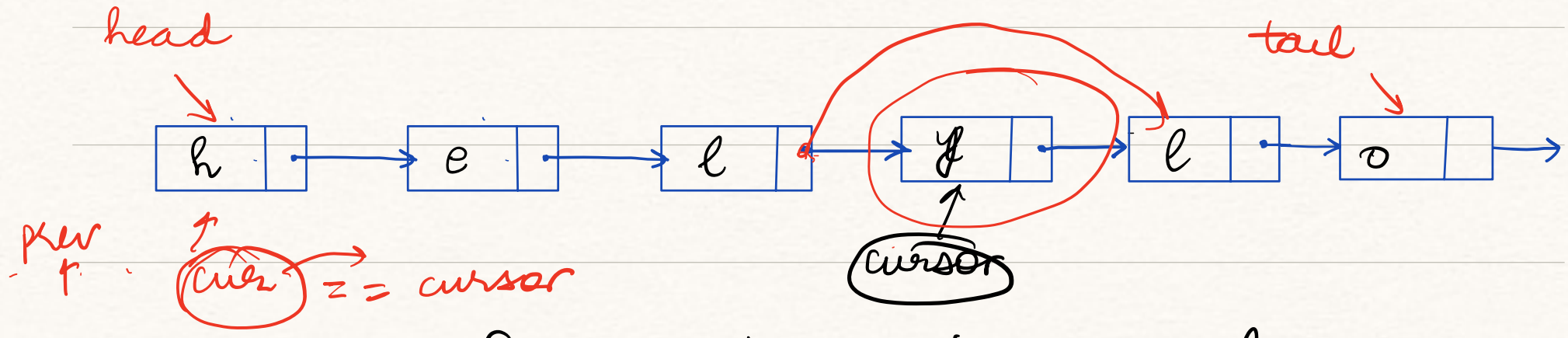
Reference to the current place /

it is a position



Claim: I can make insert/delete cheaper

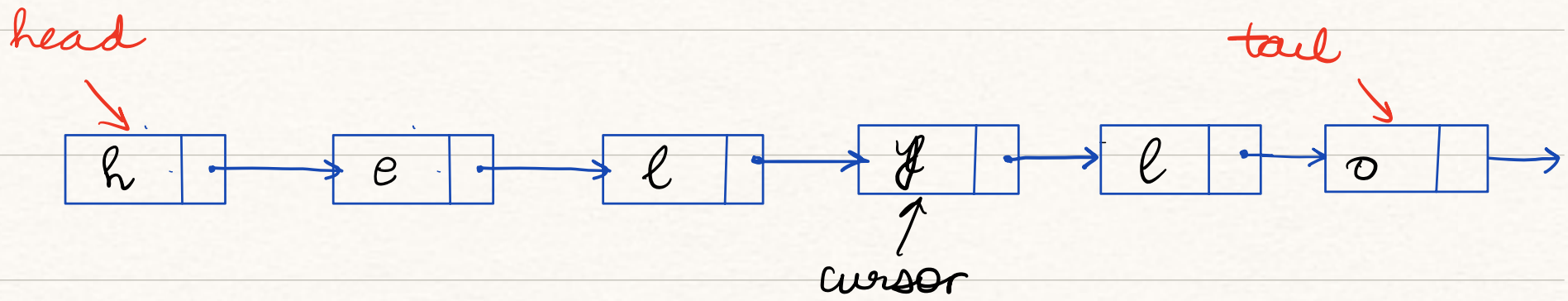
How To Delete A Node You Already Hold?



→ write a function that gives prev elem of the current element

O(i) → can I make this better

How To Delete A Node You Already Hold?

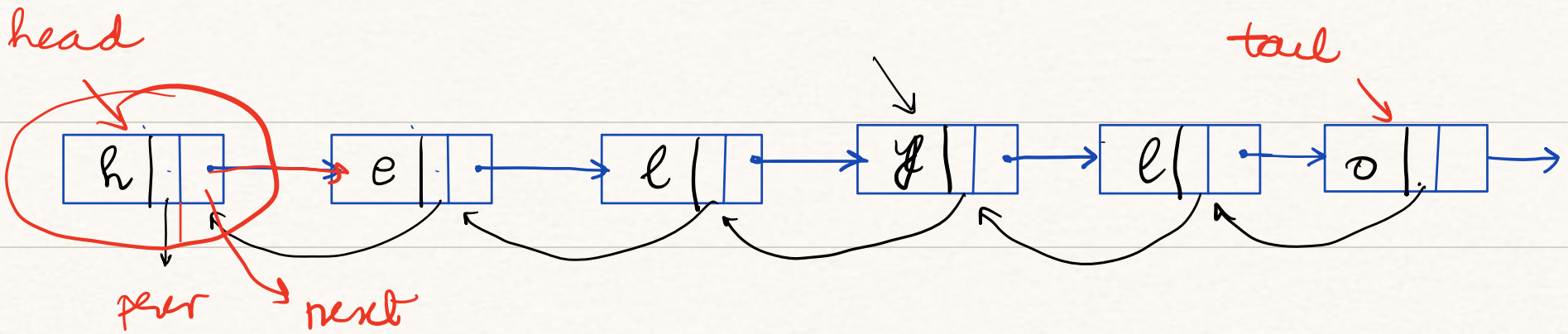


→ Need predecessor

→ Time complexity : $\Theta(n)$

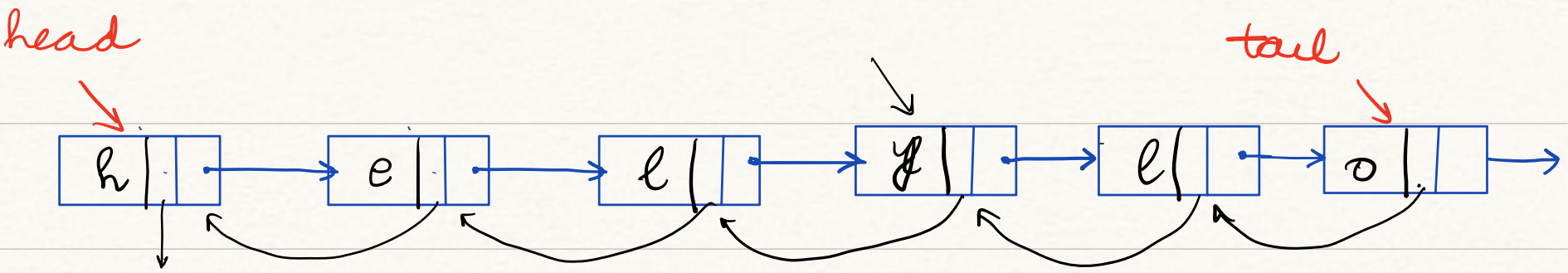
DOUBLE LINKED LIST

Each node remembers both the next and previous node



DOUBLE LINKED LIST

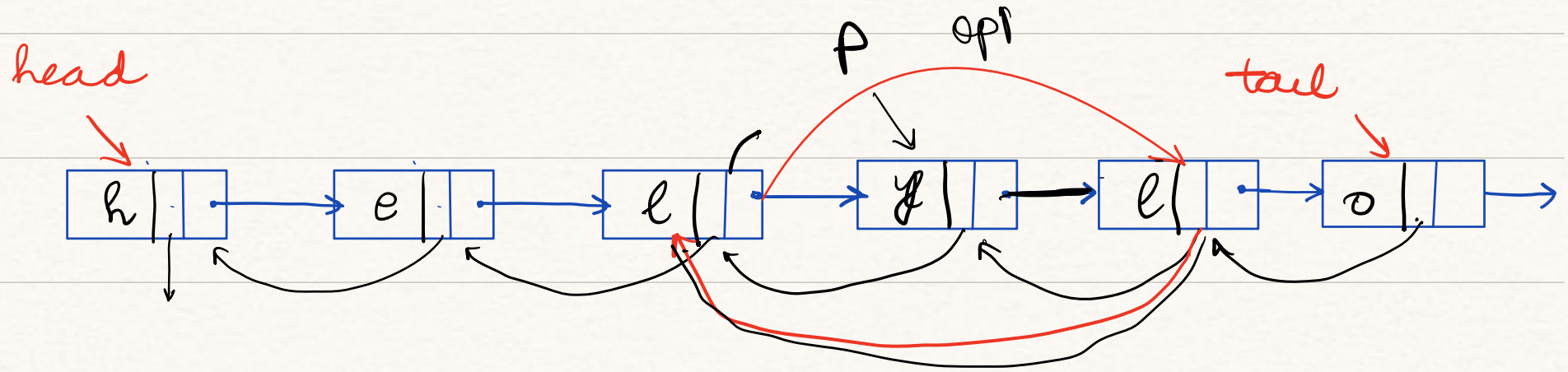
Each node remembers both the next and previous node




```
class DNode:
    def __init__(self, value, prev=None, next=None):
        self.value = value
        self.prev  = prev        # the node BEFORE this one
        self.next  = next        # the node AFTER this one
```

DOUBLE LINKED LIST

Each node remembers both the next and previous node



delete_node (p):

p.prev.next = p.next

p.next.prev = p.prev

Positional List ADT

position p - a handle on the a node

```
first() / last()              positions at the two ends
before(p) / after(p)          the neighbouring positions
add_first(e) / add_last(e) insert at an end -> new position
add_before(p,e) / add_after(p,e)   insert next to a position
replace(p, e)                 overwrite the element at p
delete(p)                     remove the element at p
```

SCORE CARD : DOUBLE LINKED LIST

first() / last()

before(p) / after(p)

add_first(p) / add_last(p)

add_before(p, e) / add_after(p, e)

replace(p, e)

delete(p)

O(1)

EXERCISE: IMPLEMENT POSITIONAL LIST ADT USING DOUBLE LINKED LIST

→ which takes more memory?

Array of n objects

LL of n objects

SCORE CARD : DOUBLE LINKED LIST

first() / last()

before(p) / after(p)

add_first(p) / add_last(p)

add_before(p, e) / add_after(p, e)

replace(p, e)

delete(p)